

64. Une méthode qui en appelle une autre

Pourquoi chaque méthode a SES deux octets, et non deux partagés.

1. Ce qu'il faut comprendre

Dans `blessier()`, l'appel nu `vivant()` veut dire « **la mienne** » : l'objet en cours est passé tel quel, comme en C++.

Et voici pourquoi chaque méthode a **ses** deux octets plutôt que deux partagés par la `struct` : si `avancer()` appelait `autre.vivant()`, un emplacement commun serait écrasé, et `avancer()` reprendrait son travail sur le mauvais ennemi. Le partage aurait marché sur les exemples simples et échoué exactement là où c'est le plus dur à voir.

La règle du non-appel-à-soi-même vaut aussi ici : **une méthode ne peut pas s'appeler elle-même**, ni par un détour. Le compilateur nomme le cycle, méthode par méthode.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1416 octets** de cartouche.

1. UN ENREGISTREMENT — ENNEMI

```
struct Ennemi {
    uint8_t x, vie;

    uint8_t vivant() { return vie > 0; }
    void blessier() { if (vivant()) vie--; }
};
```

2. UN TABLEAU — TROUPE

```
Ennemi troupe[2];
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    troupe[0].vie = 3;
    troupe[1].vie = 0;

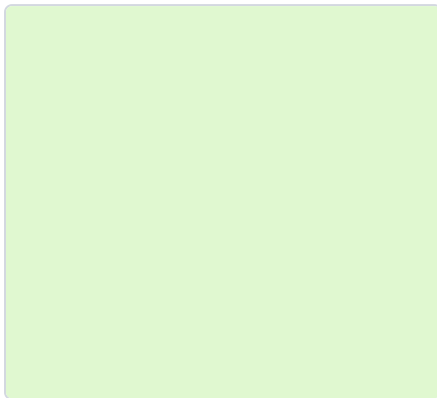
    troupe[0].blessier();
    troupe[1].blessier();

    texte(2, 6, "VIVANT MORT");
    nombre(4, 8, troupe[0].vie, 1);
    nombre(9, 8, troupe[1].vie, 1);

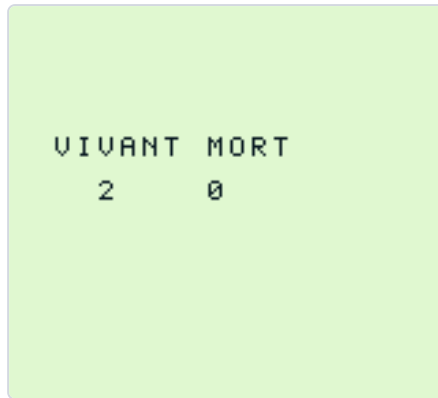
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« 2 » et « 0 » — le mort n'est pas descendu à 255.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le vivant a perdu une vie
- ✓ le mort est resté à zéro
- ✓ il n'a PAS débordé à 255
- ✓ l'appel imbriqué n'a pas perdu l'objet

Ensuite : **65. Passer d'un écran à l'autre** — niveau 9 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++